

Documentación Oficial de Jules

Módulo 3: Herramientas CLI de Jules (Jules Tools)

Google Jules Agent Docs

Fuente: <https://jules.google/docs>

Tabla de Contenido

- [1. Jules Tools Reference](#)
- [2. Practical Examples & Scripting](#)

Jules Tools Reference

Fuente: <https://jules.google/docs/cli/reference>

Jules Tools Reference

Jules Tools is a lightweight command-line interface (CLI) for interacting with Jules, Google's autonomous AI coding agent. It allows you to manage coding sessions, inspect progress, and integrate Jules into your existing development workflows and scripts directly from your terminal.

Think of Jules Tools as both a command surface and a dashboard for your coding agent, designed to keep you in your flow without needing to switch to a web browser.

Installation

[Section titled "Installation"](#)

To get started, install the tool globally using npm or pnpm.

```
npm install -g @google/jules
```

Once installed, the jules command will be available in your terminal.

Authentication

[Section titled "Authentication"](#)

Before you can use the tool, you must authenticate with your Google account.

Login

[Section titled "Login"](#)

```
jules login
```

This command will open a browser window to guide you through the Google authentication process.

Logout

[Section titled “Logout”](#)

To log out from your account:

```
jules logout
```

Usage

[Section titled “Usage”](#)

The CLI is built around commands and subcommands. You can get help for any command by using the `-h` or `—help` flag.

```
# Get general help
jules help

# Get help for a specific command (e.g., remote)
jules remote --help
```

Global Flags

[Section titled “Global Flags”](#)

- `-h` , `--help` : Displays help information for jules or a specific command.
- `--theme <string>` : Sets the theme for the terminal user interface (TUI). Options are `dark` (default) or `light` .

Example: `jules --theme light`

Available Commands

[Section titled “Available Commands”](#)

`version`

Shows the currently installed version of the Jules Tools CLI.

```
jules version
```

`remote`

The `remote` command is the primary way to interact with Jules sessions running in the cloud. It has several subcommands.

`remote list` Lists your connected repositories or active sessions.

- `--repo` : Flag to list all repositories connected to Jules.
- `--session` : Flag to list all your remote sessions.

Examples:

```
# List all connected repositories
jules remote list --repo

# List all active and past sessions
jules remote list --session
```

remote new

Creates a new remote session to delegate a task to Jules.

Jules can automatically infer the repository from your current working directory, so you can often omit the `--repo` flag.

- `--repo <repo_name>` : Specifies the repository for the session (e.g., `torvalds/linux` or `.` for the current directory's repo).
- `--session "<prompt>"` : A string describing the task for Jules to perform.
- `--parallel <number>` : Starts multiple parallel sessions to work on the same task.

Example:

```
# Start a new session to write unit tests in the 'torvalds/linux' repo
jules remote new --repo torvalds/linux --session "write unit tests"
```

remote pull

Pulls the results (e.g., code changes) from a completed session.

- `--session <session_id>` : The ID of the session you want to pull.

Example:

```
# Pull the results for session ID 123456
jules remote pull --session 123456
```

completion

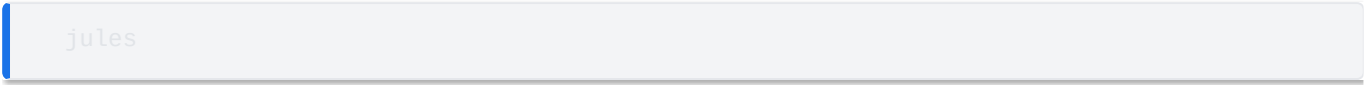
Generates an autocompletion script for your shell (e.g., `bash`, `zsh`) to enable tab completion for jules commands.

```
# Generate completion script for bash
jules completion bash
```

Interactive Dashboard (TUI)

[Section titled "Interactive Dashboard \(TUI\)"](#)

For a more interactive, visual experience, you can launch the Terminal User Interface (TUI) by running the `jules` command without any arguments.

A terminal window with a light gray background and a blue vertical bar on the left. The word "jules" is typed in a light gray font.

The TUI provides a dashboard view of your sessions, a side-by-side diff viewer for reviewing changes, and guided flows for creating new ones, similar to the web UI.

Practical Examples & Scripting

Fuente: <https://jules.google/docs/cli/examples>

Practical Examples & Scripting

Jules Tools is designed to be scriptable and can be composed with other command-line tools.

Below are some examples of Jules Tools in action:

1. Create sessions from a TODO.md file:

Assign each line item from a local TODO.md file as a new session in the current repository.

```
cat TODO.md | while IFS= read -r line; do
  jules remote new --repo . --session "$line"
done
```

2. Create a session from a GitHub Issue:

Pipe the title of the first GitHub issue assigned to you directly into a new Jules session. (Requires the gh and jq CLIs).

```
gh issue list --assignee @me --limit 1 --json title \
| jq -r '.[0].title' \
| jules remote new --repo .
```

3. Use Gemini to analyze and assign the hardest issue to Jules: Use the Gemini CLI to analyze your assigned GitHub issues, identify the most tedious one, and pipe its title to Jules.

```
gemini -p "find the most tedious issue, print it verbatim\n$(gh issue list --assignee @me)" \
| jules remote new --repo .
```